

Chapter 32 — Interval Problems

0. What You Will Learn in This Chapter

Interval problems are one of the most common interview patterns.

An interval usually represents a range:

[start, end]

or:

[start, end)

Interval problems appear in scheduling, calendars, meetings, timelines, CPU tasks, employee availability, bookings, rooms, arrows, overlaps, and range merging.

By the end of this chapter, you should understand:

- What intervals are
- Closed vs half-open intervals
- How to detect overlap
- How to merge intervals
- How to insert a new interval
- How to remove overlaps
- How to count rooms/resources
- How to find intersections
- How to handle sorted vs unsorted intervals
- How sorting by start differs from sorting by end
- How greedy interacts with intervals
- How heap interacts with intervals
- How sweep line relates to intervals
- How to avoid boundary mistakes

Core problems covered in this chapter:

1. Merge Intervals
 2. Insert Interval
 3. Non-overlapping Intervals
 4. Meeting Rooms
 5. Meeting Rooms II
 6. Minimum Number of Arrows to Burst Balloons
 7. Interval List Intersections
 8. Employee Free Time
 9. Remove Covered Intervals
 10. Merge Similar Items / Range grouping concept
 11. My Calendar I
 12. My Calendar II, sweep preview
 13. Data Stream as Disjoint Intervals
 14. Partition Labels, interval interpretation
 15. Minimum Number of Groups to Cover Intervals
-

1. What Is an Interval?

An interval represents a continuous range.

Example:

[2, 6]

means the interval starts at 2 and ends at 6.

Intervals usually appear as:

```
vector<vector<int>> intervals;
```

Example:

```
intervals = {{1, 3}, {2, 6}, {8, 10}, {15, 18}};
```

Each interval is:

[start, end]

2. Why Interval Problems Exist

Many real-world problems involve ranges:

- meeting from 10 to 11
- train from station time A to B
- balloon from x-start to x-end
- employee working from start to end
- calendar event from start to end
- booked room from start to end
- road segment from start to end
- active process from start to end

The key question is often:

How do these ranges interact?

Examples:

- Do they overlap?
- Can they be merged?
- How many overlap at once?
- What intervals are free?
- What intervals must be removed?
- What is the intersection?

3. Closed vs Half-Open Intervals

This is extremely important.

3.1 Closed Interval

[start, end]

Both start and end are included.

Example:

[1, 3] and [3, 5]

They overlap at point 3.

This is common in:

- Merge Intervals
- Minimum Arrows to Burst Balloons

3.2 Half-Open Interval

[start, end)

Start is included, end is excluded.

Example:

[1, 3) and [3, 5)

They do **not** overlap.

This is common in:

- Meeting Rooms
- My Calendar
- scheduling problems

4. Core Overlap Conditions

For two closed intervals:

[a, b]

[c, d]

They overlap if:

$\max(a, c) \leq \min(b, d)$

or equivalently:

$c \leq b \ \&\& \ a \leq d$

If intervals are sorted by start and $a \leq c$, then overlap becomes:

$c \leq b$

For half-open intervals:

[a, b)

[c, d)

They overlap if:

$\max(a, c) < \min(b, d)$

If sorted by start:

$c < b$

5. Core Need / Why Intervals Need Special Handling

Intervals are not single values.

They have:

start

end

length
overlap relationship
ordering

A lot of interval problems become easy only after sorting.

For example:

Unsorted:

[8,10], [1,3], [2,6], [15,18]

Hard to merge.

Sorted by start:

[1,3], [2,6], [8,10], [15,18]

Now overlapping intervals are adjacent.

6. When to Use Interval Patterns

Use interval patterns when the problem mentions:

6.1 Ranges

Keywords:

- interval
- range
- segment
- start/end
- from/to
- booking
- meeting
- schedule

6.2 Overlap

Keywords:

- overlapping intervals
- conflict
- collision
- intersection
- common time

6.3 Merge

Keywords:

- merge intervals
- combine ranges
- union of intervals

6.4 Remove or Minimize Conflicts

Keywords:

- minimum removals
- non-overlapping
- erase overlap
- avoid conflict

6.5 Count Simultaneous Active Intervals

Keywords:

- minimum rooms
 - maximum overlap
 - groups
 - platforms
 - active meetings
-

7. When NOT to Use Interval Patterns

Do not force interval logic when:

7.1 Data Is Not a Range

If the problem only has points, use sorting/hash/math unless ranges are implied.

7.2 Need Shortest Path

Use BFS/Dijkstra.

7.3 Need Dynamic Range Updates

Use difference array, Fenwick tree, segment tree, or sweep line.

7.4 Need Exact Lookup

Use hash map or set.

8. Mental Models

8.1 Timeline Model

Intervals are blocks on a timeline.

Two intervals overlap if their blocks touch or intersect, depending on closed/half-open rules.

8.2 Current Merged Block

For merge problems, maintain one current interval.

If next interval overlaps, extend current.

If not, push current and start a new block.

8.3 Earliest Ending Leaves Most Room

For removal/scheduling problems, keep intervals that end earliest.

This leaves maximum space for future intervals.

8.4 Active Meetings

For room-count problems, keep track of currently active intervals.

If a meeting starts before another ends, both need separate rooms.

9. Pattern Recognition Checklist

Ask:

16. Does the problem contain [start, end] pairs?
17. Do intervals need sorting?
18. Are intervals closed or half-open?
19. Does the problem ask to merge?
20. Does the problem ask to insert?
21. Does the problem ask to remove overlaps?

- 22. Does the problem ask for intersections?
 - 23. Does the problem ask for minimum rooms/groups?
 - 24. Does sorting by start help?
 - 25. Does sorting by end help?
 - 26. Do I need a heap for active intervals?
 - 27. Do I need sweep line for max overlap?
-

10. C++ Tools Needed

10.1 Sort Intervals by Start

```
sort(intervals.begin(), intervals.end());
```

Default vector sorting sorts by:

first element, then second element

So for intervals:

```
[start, end]
```

it sorts by start, then end.

10.2 Sort Intervals by End

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

Used for greedy scheduling.

10.3 Min Heap of End Times

```
priority_queue<int, vector<int>, greater<int>> minHeap;
```

Used for Meeting Rooms II.

10.4 Pair / Vector Access

```
int start = interval[0];  
int end = interval[1];
```

11. Core Templates

Template 1 — Merge Intervals

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {  
    if (intervals.empty()) {  
        return {};  
    }  
  
    sort(intervals.begin(), intervals.end());
```

```

vector<vector<int>> result;

for (auto& interval : intervals) {
    if (result.empty() || interval[0] > result.back()[1]) {
        result.push_back(interval);
    } else {
        result.back()[1] = max(result.back()[1], interval[1]);
    }
}

return result;
}

```

Template 2 — Insert Interval

```

vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>&
newInterval) {
    vector<vector<int>> result;

    int i = 0;
    int n = intervals.size();

    while (i < n && intervals[i][1] < newInterval[0]) {
        result.push_back(intervals[i]);
        i++;
    }

    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = min(newInterval[0], intervals[i][0]);
        newInterval[1] = max(newInterval[1], intervals[i][1]);
        i++;
    }

    result.push_back(newInterval);

    while (i < n) {
        result.push_back(intervals[i]);
        i++;
    }

    return result;
}

```

Template 3 — Non-overlapping Intervals

```
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    if (intervals.empty()) {
        return 0;
    }

    sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
        return a[1] < b[1];
    });

    int removals = 0;
    int lastEnd = intervals[0][1];

    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] < lastEnd) {
            removals++;
        } else {
            lastEnd = intervals[i][1];
        }
    }

    return removals;
}
```

Template 4 — Meeting Rooms II With Heap

```
int minMeetingRooms(vector<vector<int>>& intervals) {
    if (intervals.empty()) {
        return 0;
    }

    sort(intervals.begin(), intervals.end());

    priority_queue<int, vector<int>, greater<int>> minHeap;

    for (auto& interval : intervals) {
        int start = interval[0];
        int end = interval[1];

        if (!minHeap.empty() && minHeap.top() <= start) {
```



```

        minHeap.pop();
    }

    minHeap.push(end);
}

return minHeap.size();
}

```

Template 5 — Interval Intersection

```

vector<vector<int>> intervalIntersection(vector<vector<int>>& firstList,
                                       vector<vector<int>>& secondList) {
    vector<vector<int>> result;

    int i = 0;
    int j = 0;

    while (i < firstList.size() && j < secondList.size()) {
        int start = max(firstList[i][0], secondList[j][0]);
        int end = min(firstList[i][1], secondList[j][1]);

        if (start <= end) {
            result.push_back({start, end});
        }

        if (firstList[i][1] < secondList[j][1]) {
            i++;
        } else {
            j++;
        }
    }

    return result;
}

```

12. Major Interval Variations

Merge intervals	sort by start, combine overlaps	Merge Intervals
Insert interval	before / overlapping / after	Insert Interval

Variation	Core Idea	Example
Remove overlaps	sort by end, keep earliest ending	Non-overlapping Intervals
Meeting conflict	sort by start, compare adjacent	Meeting Rooms
Minimum rooms	min heap of end times	Meeting Rooms II
Arrows / coverage	sort by end, one point covers many	Minimum Arrows
Intersections	two sorted lists, move earlier end	Interval List Intersections
Free time	merge busy intervals, gaps are free	Employee Free Time
Covered intervals	sort start asc, end desc	Remove Covered Intervals
Calendar booking	ordered map by start	My Calendar I
Max overlap	sweep line	My Calendar III / Groups

13. Full Problem Walkthrough 1 — LeetCode 56: Merge Intervals

13.1 Problem Summary

Given intervals, merge all overlapping intervals.

Example:

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
```

Output:

```
[[1,6], [8,10], [15,18]]
```

13.2 How to Identify This Pattern

Problem directly says:

```
merge intervals
```

Sort by start.

Then scan and merge overlapping intervals.

13.3 Brute Force Idea

Compare every interval with every other interval.

This is complicated and can become $O(n^2)$.

13.4 Optimized Intuition

After sorting by start, overlapping intervals appear next to each other.

Maintain result.

For each interval:

- if no overlap with last result interval, push it
- otherwise extend last result interval

13.5 C++ Code

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        if (intervals.empty()) {
            return {};
        }

        sort(intervals.begin(), intervals.end());

        vector<vector<int>> result;

        for (auto& interval : intervals) {
            if (result.empty() || interval[0] > result.back()[1]) {
                result.push_back(interval);
            } else {
                result.back()[1] = max(result.back()[1], interval[1]);
            }
        }

        return result;
    }
};
```

13.6 Dry Run

Input:

[[1,3], [2,6], [8,10], [15,18]]

[1,3]	[]	push	[[1,3]]
[2,6]	[[1,3]]	merge	[[1,6]]
[8,10]	[[1,6]]	push	[[1,6],[8,10]]
[15,18]	[[1,6],[8,10]]	push	[[1,6],[8,10],[15,18]]

13.7 Correctness Explanation

After sorting, if an interval does not overlap the current merged interval, no later interval can overlap the current merged interval because later intervals start even later.

So it is safe to finalize the current interval.

13.8 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

for output.

13.9 Common Mistakes

- Not sorting first
- Forgetting to update end with max
- Wrong overlap condition
- Forgetting empty input

14. Full Problem Walkthrough 2 — LeetCode 57: Insert Interval

14.1 Problem Summary

Given sorted non-overlapping intervals and a new interval.

Insert new interval and merge if needed.

Example:

`intervals = [[1,3], [6,9]]`

`newInterval = [2,5]`

Output:

`[[1,5], [6,9]]`

14.2 How to Identify This Pattern

Problem says:

- intervals are sorted
- non-overlapping
- insert new interval

Use three phases:

28. intervals before new interval
29. intervals overlapping new interval
30. intervals after new interval

14.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>&
newInterval) {
```

```

vector<vector<int>> result;

int i = 0;
int n = intervals.size();

while (i < n && intervals[i][1] < newInterval[0]) {
    result.push_back(intervals[i]);
    i++;
}

while (i < n && intervals[i][0] <= newInterval[1]) {
    newInterval[0] = min(newInterval[0], intervals[i][0]);
    newInterval[1] = max(newInterval[1], intervals[i][1]);
    i++;
}

result.push_back(newInterval);

while (i < n) {
    result.push_back(intervals[i]);
    i++;
}

return result;
}
};

```

14.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

14.5 Common Mistakes

- Sorting unnecessarily
 - Forgetting to push merged new interval
 - Wrong overlap condition
 - Not handling insertion before all or after all intervals
-

15. Full Problem Walkthrough 3 — LeetCode 435: Non-overlapping Intervals

15.1 Problem Summary

Given intervals, return minimum number to remove so remaining intervals do not overlap.

Example:

```
[[1,2], [2,3], [3,4], [1,3]]
```

Output:

```
1
```

Remove [1,3].

15.2 How to Identify This Pattern

Minimum removals means:

maximize number of intervals kept

Choose intervals ending earliest.

15.3 C++ Code

```
class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.empty()) {
            return 0;
        }

        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
            return a[1] < b[1];
        });

        int removals = 0;
        int lastEnd = intervals[0][1];

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] < lastEnd) {
                removals++;
            } else {
                lastEnd = intervals[i][1];
            }
        }
    }
};
```

```
    }  
  
    return removals;  
}  
};
```

15.4 Important Boundary Rule

For this problem:

[1,2] and [2,3]

do **not** overlap.

So overlap is:

`intervals[i][0] < lastEnd`

not:

`intervals[i][0] <= lastEnd`

15.5 Complexity

Time:

$O(n \log n)$

Space:

$O(1)$ extra

16. Full Problem Walkthrough 4 — Meeting Rooms

16.1 Problem Summary

Given meeting intervals, determine if a person can attend all meetings.

Example:

`[[0,30], [5,10], [15,20]]`

Output:

false

16.2 How to Identify This Pattern

Need detect overlap.

Sort by start and compare adjacent intervals.

16.3 C++ Code

```
bool canAttendMeetings(vector<vector<int>>& intervals) {  
    sort(intervals.begin(), intervals.end());  
  
    for (int i = 1; i < intervals.size(); i++) {  
        if (intervals[i][0] < intervals[i - 1][1]) {
```

```
        return false;
    }
}

return true;
}
```

16.4 Complexity

Time:

$O(n \log n)$

Space:

$O(1)$

16.5 Common Mistakes

- Treating touching intervals as overlap
- Not sorting
- Comparing non-adjacent intervals unnecessarily

17. Full Problem Walkthrough 5 — LeetCode 253: Meeting Rooms II

17.1 Problem Summary

Given meeting intervals, return minimum number of rooms required.

Example:

[[0,30], [5,10], [15,20]]

Output:

2

17.2 How to Identify This Pattern

Need maximum number of overlapping meetings.

Use min heap of ending times.

17.3 C++ Code

```
int minMeetingRooms(vector<vector<int>>& intervals) {
    if (intervals.empty()) {
        return 0;
    }

    sort(intervals.begin(), intervals.end());

    priority_queue<int, vector<int>, greater<int>> minHeap;

    for (auto& meeting : intervals) {
```



```

int start = meeting[0];
int end = meeting[1];

if (!minHeap.empty() && minHeap.top() <= start) {
    minHeap.pop();
}

minHeap.push(end);
}

return minHeap.size();
}

```

17.4 Dry Run

Input:

[[0,30], [5,10], [15,20]]

[0,30]	[]	add room ending 30	[30]
[5,10]	[30]	need new room	[10,30]
[15,20]	[10,30]	reuse room ending 10	[20,30]

Answer:

2

17.5 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

18. Full Problem Walkthrough 6 — LeetCode 452: Minimum Number of Arrows to Burst Balloons

18.1 Problem Summary

Each balloon is an interval [start, end].

One arrow shot at position x bursts every balloon where:

$\text{start} \leq x \leq \text{end}$

Return minimum arrows.

18.2 How to Identify This Pattern

This is interval coverage.

Sort by end.

Shoot arrow at earliest end.

18.3 C++ Code

```
class Solution {
public:
    int findMinArrowShots(vector<vector<int>>& points) {
        if (points.empty()) {
            return 0;
        }

        sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>&
b) {
            return a[1] < b[1];
        });

        int arrows = 1;
        long long arrowPos = points[0][1];

        for (int i = 1; i < points.size(); i++) {
            if (points[i][0] > arrowPos) {
                arrows++;
                arrowPos = points[i][1];
            }
        }

        return arrows;
    }
};
```

18.4 Important Boundary Rule

If:

balloon start == arrow position

the arrow bursts it.

So new arrow is needed only when:

$\text{points}[i][0] > \text{arrowPos}$

18.5 Complexity

Time:

$O(n \log n)$

Space:

$O(1)$

19. Full Problem Walkthrough 7 — LeetCode 986: Interval List Intersections

19.1 Problem Summary

Given two sorted lists of disjoint intervals, return their intersections.

Example:

```
firstList = [[0,2], [5,10], [13,23], [24,25]]
secondList = [[1,5], [8,12], [15,24], [25,26]]
```

Output:

```
[[1,2], [5,5], [8,10], [15,23], [24,24], [25,25]]
```

19.2 How to Identify This Pattern

Need intersection of two sorted interval lists.

Use two pointers.

19.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> intervalIntersection(vector<vector<int>>& firstList,
                                           vector<vector<int>>& secondList) {
        vector<vector<int>> result;

        int i = 0;
        int j = 0;

        while (i < firstList.size() && j < secondList.size()) {
            int start = max(firstList[i][0], secondList[j][0]);
            int end = min(firstList[i][1], secondList[j][1]);

            if (start <= end) {
                result.push_back({start, end});
            }

            if (firstList[i][1] < secondList[j][1]) {
                i++;
            } else {
                j++;
            }
        }

        return result;
    }
};
```

```

    }
}

return result;
}
};

```

19.4 Why Move the Earlier Ending Interval?

The interval that ends earlier cannot intersect future intervals from the other list anymore.

So move that pointer.

19.5 Complexity

Time:

$O(n + m)$

Space:

$O(1)$ extra

excluding output.

20. Full Problem Walkthrough 8 — Employee Free Time

20.1 Problem Summary

Given each employee's schedule of working intervals, return common free time intervals.

Example:

```

schedule = [
  [[1,2], [5,6]],
  [[1,3]],
  [[4,10]]
]

```

Merged busy intervals:

[1,3], [4,10]

Free time:

[3,4]

20.2 How to Identify This Pattern

Common free time is the gap between merged busy intervals.

Steps:

31. Flatten all intervals
32. Sort by start
33. Merge busy intervals
34. Return gaps

20.3 C++ Code

```
vector<vector<int>> employeeFreeTime(vector<vector<vector<int>>>&
schedule) {
    vector<vector<int>> intervals;

    for (auto& employee : schedule) {
        for (auto& interval : employee) {
            intervals.push_back(interval);
        }
    }

    sort(intervals.begin(), intervals.end());

    vector<vector<int>> freeTime;

    int currentStart = intervals[0][0];
    int currentEnd = intervals[0][1];

    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] <= currentEnd) {
            currentEnd = max(currentEnd, intervals[i][1]);
        } else {
            freeTime.push_back({currentEnd, intervals[i][0]});
            currentStart = intervals[i][0];
            currentEnd = intervals[i][1];
        }
    }

    return freeTime;
}
```

20.4 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

21. Full Problem Walkthrough 9 — LeetCode 1288: Remove Covered Intervals

21.1 Problem Summary

An interval $[a,b]$ is covered by $[c,d]$ if:

$c \leq a$ and $b \leq d$

Return number of remaining intervals after removing covered intervals.

21.2 How to Identify This Pattern

Need detect intervals contained inside previous intervals.

Sort by:

start ascending
end descending

Why end descending?

For same start, larger interval should come first.

21.3 C++ Code

```
class Solution {
public:
    int removeCoveredIntervals(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
            if (a[0] != b[0]) {
                return a[0] < b[0];
            }

            return a[1] > b[1];
        });

        int remaining = 0;
        int maxEnd = 0;

        for (auto& interval : intervals) {
            if (interval[1] > maxEnd) {
                remaining++;
                maxEnd = interval[1];
            }
        }

        return remaining;
    }
};
```

```
}  
};
```

21.4 Complexity

Time:

$O(n \log n)$

Space:

$O(1)$

21.5 Common Mistake

Sorting same starts by end ascending can count covered intervals incorrectly.

22. Full Problem Walkthrough 10 — LeetCode 729: My Calendar I

22.1 Problem Summary

Book intervals [start, end).

Return false if new booking overlaps existing booking.

22.2 How to Identify This Pattern

Need dynamic interval overlap detection.

Use ordered map:

start -> end

For new interval, only previous and next intervals can overlap.

22.3 C++ Code

```
class MyCalendar {  
private:  
    map<int, int> intervals;  
  
public:  
    MyCalendar() {  
    }  
  
    bool book(int start, int end) {  
        auto next = intervals.lower_bound(start);  
  
        if (next != intervals.end() && next->first < end) {  
            return false;  
        }  
  
        if (next != intervals.begin()) {  
            auto prevIt = prev(next);  

```

```

        if (prevIt->second > start) {
            return false;
        }
    }

    intervals[start] = end;

    return true;
}
};

```

22.4 Complexity

Each booking:

$O(\log n)$

Space:

$O(n)$

23. Full Problem Walkthrough 11 — LeetCode 2406: Divide Intervals Into Minimum Number of Groups

23.1 Problem Summary

Given intervals, divide them into minimum number of groups so no intervals in same group overlap.

Equivalent:

Find maximum number of overlapping intervals.

23.2 How to Identify This Pattern

Minimum groups needed equals maximum simultaneous active intervals.

Use min heap of group end times.

Intervals are closed, so $[1,5]$ and $[5,8]$ overlap.

A group can be reused only if:

$\text{previousEnd} < \text{currentStart}$

23.3 C++ Code

```

class Solution {
public:
    int minGroups(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
    }
};

```



```

priority_queue<int, vector<int>, greater<int>> minHeap;

for (auto& interval : intervals) {
    int start = interval[0];
    int end = interval[1];

    if (!minHeap.empty() && minHeap.top() < start) {
        minHeap.pop();
    }

    minHeap.push(end);
}

return minHeap.size();
}
};

```

23.4 Important Boundary Rule

Intervals are closed.

So:

[1,5] and [5,8]

overlap.

Need:

$\text{minHeap.top()} < \text{start}$

not:

$\text{minHeap.top()} \leq \text{start}$

23.5 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

24. Full Problem Walkthrough 12 — Data Stream as Disjoint Intervals

24.1 Problem Summary

Given a stream of integers, maintain disjoint intervals.

Operations:

```
addNum(value)
getIntervals()
```

24.2 How to Identify This Pattern

Need dynamically merge neighboring intervals.

Use ordered map:

```
start -> end
```

24.3 C++ Code

```
class SummaryRanges {
private:
    map<int, int> intervals;

public:
    SummaryRanges() {
    }

    void addNum(int value) {
        auto next = intervals.upper_bound(value);

        int start = value;
        int end = value;

        if (next != intervals.begin()) {
            auto prevIt = prev(next);

            if (prevIt->second >= value) {
                return;
            }

            if (prevIt->second + 1 == value) {
                start = prevIt->first;
                intervals.erase(prevIt);
            }
        }

        if (next != intervals.end() && next->first == value + 1) {
            end = next->second;
            intervals.erase(next);
        }

        intervals[start] = end;
    }
}
```

```

vector<vector<int>> getIntervals() {
    vector<vector<int>> result;

    for (auto& [start, end] : intervals) {
        result.push_back({start, end});
    }

    return result;
}
};

```

24.4 Complexity

addNum:

$O(\log n)$

getIntervals:

$O(\text{number of intervals})$

25. Full Problem Walkthrough 13 — Partition Labels as Intervals

25.1 Problem Summary

Partition string so each character appears in at most one part.

25.2 Interval Interpretation

Each character creates an interval:

[first occurrence, last occurrence]

Then the problem becomes merging overlapping character intervals.

25.3 C++ Code

```

class Solution {
public:
    vector<int> partitionLabels(string s) {
        vector<int> last(26, 0);

        for (int i = 0; i < s.size(); i++) {
            last[s[i] - 'a'] = i;
        }

        vector<int> result;

        int start = 0;

```

```

int end = 0;

for (int i = 0; i < s.size(); i++) {
    end = max(end, last[s[i] - 'a']);

    if (i == end) {
        result.push_back(end - start + 1);
        start = i + 1;
    }
}

return result;
}
};

```

25.4 Key Lesson

Some problems do not explicitly give intervals, but intervals are hidden in the structure.

26. Full Problem Walkthrough 14 — Meeting Rooms II With Sweep Line

26.1 Problem Summary

Return minimum rooms required for meetings.

26.2 Sweep Line Idea

Each meeting has:

```

start event: +1
end event: -1

```

Sort events.

At each time, active meetings determine required rooms.

For half-open meetings [start, end), end should be processed before start at same time.

26.3 C++ Code

```

int minMeetingRooms(vector<vector<int>>& intervals) {
    vector<pair<int, int>> events;

    for (auto& interval : intervals) {
        events.push_back({interval[0], 1});
        events.push_back({interval[1], -1});
    }

    sort(events.begin(), events.end());
}

```

```
int active = 0;
int rooms = 0;

for (auto& [time, delta] : events) {
    active += delta;
    rooms = max(rooms, active);
}

return rooms;
}
```

26.4 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

27. Full Problem Walkthrough 15 — My Calendar II Preview

27.1 Problem Summary

Allow double bookings but not triple bookings.

27.2 Interval / Sweep Line Idea

Use ordered map events:

```
events[start]++;
events[end]--;
```

After adding booking, scan active count.

If active reaches 3, undo.

27.3 C++ Code

```
class MyCalendarTwo {
private:
    map<int, int> events;

public:
    MyCalendarTwo() {
    }

    bool book(int start, int end) {
        events[start]++;
```

```
events[end]--;

int active = 0;

for (auto& [time, delta] : events) {
    active += delta;

    if (active >= 3) {
        events[start]--;
        events[end]++;

        if (events[start] == 0) {
            events.erase(start);
        }

        if (events[end] == 0) {
            events.erase(end);
        }

        return false;
    }
}

return true;
};
```

27.4 Complexity

Each booking:

$O(n)$

because of scanning events.

28. Common Interval Edge Cases

28.1 Empty Intervals

Return empty result or 0 depending on problem.

28.2 One Interval

Usually directly returned or answer 1.

28.3 Touching Intervals

Know whether touching means overlap.

Closed:

[1,3] and [3,5] overlap

Half-open:

[1,3) and [3,5) do not overlap

28.4 Contained Intervals

Example:

[1,10], [2,3]

Merge should keep [1,10].

Remove covered should remove [2,3].

28.5 Duplicate Intervals

Sorting handles them, but be aware in covered interval problems.

28.6 Large Coordinates

Use long long if coordinate comparisons or differences can overflow.

29. Common Mistakes and Wrong Code

Mistake 1 — Wrong Overlap Condition

Closed intervals:

start <= end

Half-open intervals:

start < end

Mistake 2 — Sorting by Wrong Key

Merge intervals:

sort by start

Remove overlaps:

sort by end

Remove covered:

start ascending, end descending

Mistake 3 — Forgetting Max End During Merge

Wrong:

```
result.back()[1] = interval[1];
```

Correct:

```
result.back()[1] = max(result.back()[1], interval[1]);
```

Mistake 4 — Treating Meeting End as Inclusive

Meetings are usually half-open.

So if one ends at 10 and another starts at 10, no conflict.

Mistake 5 — Not Handling Empty Input

Do not access:

intervals[0]

before checking empty.

30. Interview Explanation Templates

30.1 Merge Intervals

I sort intervals by start time. Then I scan from left to right. If the current interval overlaps the last merged interval, I extend the end. Otherwise, I start a new merged interval.

30.2 Insert Interval

Because intervals are already sorted and non-overlapping, I divide them into three groups: intervals before the new interval, intervals overlapping it, and intervals after it.

30.3 Non-overlapping Intervals

I sort by end time and greedily keep intervals that finish earliest. This leaves maximum room for later intervals.

30.4 Meeting Rooms II

I sort meetings by start time and use a min heap of end times. If the earliest ending meeting ends before the current meeting starts, I reuse that room. Otherwise, I allocate a new room.

30.5 Interval Intersection

For two sorted interval lists, I compare the current intervals. Their intersection is from max starts to min ends. Then I advance the interval that ends earlier.

31. Complexity Patterns

Merge Intervals	$O(n \log n)$	$O(n)$
Insert Interval	$O(n)$	$O(n)$
Non-overlapping Intervals	$O(n \log n)$	$O(1)$
Meeting Rooms	$O(n \log n)$	$O(1)$
Meeting Rooms II Heap	$O(n \log n)$	$O(n)$
Minimum Arrows	$O(n \log n)$	$O(1)$
Interval Intersections	$O(n + m)$	$O(1)$ extra
Employee Free Time	$O(n \log n)$	$O(n)$

Problem Type	Time	Space
Remove Covered Intervals	$O(n \log n)$	$O(1)$
My Calendar I	$O(\log n)$ per booking	$O(n)$
My Calendar II	$O(n)$ per booking	$O(n)$
Disjoint Intervals Stream	$O(\log n)$ add	$O(n)$

32. Time-Boxed Practice Guidance

Easy Interval Problems

Target:

10–20 minutes

Examples:

- Meeting Rooms
- Merge Intervals
- Insert Interval

Medium Interval Problems

Target:

25–45 minutes

Examples:

- Non-overlapping Intervals
- Minimum Arrows
- Meeting Rooms II
- Interval List Intersections
- Remove Covered Intervals

Harder Interval Problems

Target:

45–75 minutes

Examples:

- Employee Free Time
- My Calendar I
- My Calendar II
- Data Stream as Disjoint Intervals
- Range Module

33. Expanded Practice Set

33.1 Must-Do Easy / Intro

35. Meeting Rooms
36. Merge Intervals
37. Insert Interval
38. Summary Ranges

- 39. Can Attend Meetings

33.2 Must-Do Medium

- 40. Non-overlapping Intervals
- 41. Minimum Number of Arrows to Burst Balloons
- 42. Meeting Rooms II
- 43. Interval List Intersections
- 44. Remove Covered Intervals
- 45. Partition Labels
- 46. Divide Intervals Into Minimum Number of Groups
- 47. My Calendar I
- 48. Employee Free Time

33.3 Strong Interview Practice

- 49. My Calendar II
- 50. My Calendar III
- 51. Data Stream as Disjoint Intervals
- 52. Range Module
- 53. Amount of New Area Painted Each Day
- 54. Car Pooling
- 55. Describe the Painting
- 56. Number of Flowers in Full Bloom

33.4 Hard / Stretch

- 57. Range Module
 - 58. Employee Free Time
 - 59. My Calendar III
 - 60. The Skyline Problem
 - 61. Rectangle Area II
 - 62. Count Integers in Intervals
 - 63. Falling Squares
-

34. Suggested Study Order

Step 1 — Basic Merge

Start with:

- 64. Merge Intervals
- 65. Insert Interval
- 66. Summary Ranges

Goal:

Learn sorting by start and merging.

Step 2 — Conflict Detection

Solve:

- 67. Meeting Rooms
- 68. Non-overlapping Intervals

Goal:

Learn overlap detection and interval removal.

Step 3 — Resource Counting

Solve:

- 69. Meeting Rooms II
- 70. Divide Intervals Into Minimum Number of Groups

Goal:

Learn heap/sweep for active intervals.

Step 4 — Coverage and Intersections

Solve:

- 71. Minimum Arrows
- 72. Interval List Intersections
- 73. Employee Free Time

Goal:

Learn coverage and gap logic.

Step 5 — Dynamic Intervals

Solve:

- 74. My Calendar I
- 75. My Calendar II
- 76. Data Stream as Disjoint Intervals
- 77. Range Module

Goal:

Learn ordered maps and dynamic interval management.

35. Pattern Decision Table

merge overlapping ranges	sort by start	Merge Intervals
insert one interval	three phases	Insert Interval
remove minimum overlaps	sort by end	Non-overlapping
can attend all meetings	compare adjacent	Meeting Rooms
minimum rooms/groups	heap or sweep	Meeting Rooms II
one point covers intervals	sort by end	Arrows
two sorted interval lists	two pointers	Intersections
common free time	merge busy, find gaps	Employee Free Time
interval contained in another	start asc, end desc	Remove Covered
dynamic bookings	ordered map	My Calendar
max overlap	sweep line	My Calendar III
stream of numbers into intervals	ordered map merge	Summary Ranges

36. Mini Problem 1 — Summary Ranges

Pattern

Convert sorted array into disjoint ranges.

```
class Solution {
public:
    vector<string> summaryRanges(vector<int>& nums) {
        vector<string> result;

        int i = 0;

        while (i < nums.size()) {
            int start = nums[i];

            while (i + 1 < nums.size() && nums[i + 1] == nums[i] + 1) {
                i++;
            }

            int end = nums[i];

            if (start == end) {
                result.push_back(to_string(start));
            } else {
                result.push_back(to_string(start) + "->" + to_string(end));
            }

            i++;
        }

        return result;
    }
};
```

37. Mini Problem 2 — Car Pooling

Pattern

Difference array / sweep line interval capacity.

```
class Solution {
public:
```

```

bool carPooling(vector<vector<int>>& trips, int capacity) {
    vector<int> diff(1001, 0);

    for (auto& trip : trips) {
        int passengers = trip[0];
        int from = trip[1];
        int to = trip[2];

        diff[from] += passengers;
        diff[to] -= passengers;
    }

    int active = 0;

    for (int x : diff) {
        active += x;

        if (active > capacity) {
            return false;
        }
    }

    return true;
}
};

```

38. Mini Problem 3 — Meeting Rooms II With Two Sorted Arrays

Pattern

Separate start and end times.

```

int minMeetingRooms(vector<vector<int>>& intervals) {
    vector<int> starts;
    vector<int> ends;

    for (auto& interval : intervals) {
        starts.push_back(interval[0]);
        ends.push_back(interval[1]);
    }

    sort(starts.begin(), starts.end());

```

```

sort(ends.begin(), ends.end());

int rooms = 0;
int maxRooms = 0;
int i = 0;
int j = 0;

while (i < starts.size()) {
    if (starts[i] < ends[j]) {
        rooms++;
        maxRooms = max(maxRooms, rooms);
        i++;
    } else {
        rooms--;
        j++;
    }
}

return maxRooms;
}

```

39. Mini Problem 4 — Find Gaps After Merging

```

vector<vector<int>> findGaps(vector<vector<int>>& intervals) {
    vector<vector<int>> merged = merge(intervals);
    vector<vector<int>> gaps;

    for (int i = 1; i < merged.size(); i++) {
        gaps.push_back({merged[i - 1][1], merged[i][0]});
    }

    return gaps;
}

```

Used in:

- Employee Free Time
- free calendar slots
- available meeting time

40. Common Edge Case Checklist

Before submitting interval code, test:

```
[]  
[[1,2]]  
[[1,3],[2,6]]  
[[1,4],[4,5]]  
[[1,4],[2,3]]  
[[1,10],[2,3],[4,5]]  
[[1,2],[3,4]]  
same start different end  
same end different start  
negative coordinates  
very large coordinates
```

41. What to Master Before Chapter 33

Before moving to Sweep Line, make sure you can:

- Detect interval overlap
- Distinguish closed vs half-open intervals
- Sort intervals by start
- Sort intervals by end
- Merge intervals
- Insert an interval
- Remove overlapping intervals
- Solve Meeting Rooms
- Solve Meeting Rooms II
- Solve Minimum Arrows
- Solve Interval List Intersections
- Solve Remove Covered Intervals
- Understand heap for active intervals
- Understand sweep-line connection
- Handle boundary equality correctly

42. Final Cheat Sheet

Overlap, Closed Intervals

```
max(aStart, bStart) <= min(aEnd, bEnd)
```

Overlap, Half-Open Intervals

```
max(aStart, bStart) < min(aEnd, bEnd)
```

Merge Condition, Closed Intervals

```
interval[0] <= result.back()[1]
```

Non-overlap, Meeting Intervals

```
currentStart >= previousEnd
```

Sort by Start

```
sort(intervals.begin(), intervals.end());
```

Sort by End

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

Min Heap of End Times

```
priority_queue<int, vector<int>, greater<int>> minHeap;
```

43. Core Takeaway

Interval problems are timeline problems.

The core questions are:

Do these intervals overlap?
Should I merge them?
Should I keep or remove one?
How many are active at the same time?
Where are the gaps?

Most interval problems become easier after sorting.

Use:

- sort by start for merging
- sort by end for greedy removal/coverage
- heap for active intervals
- two pointers for intersections
- ordered map for dynamic intervals
- sweep line for maximum overlap

The biggest danger is boundary handling.

Always clarify:

Is the interval closed [start, end] or half-open [start, end)?

That one detail changes the overlap condition.